2018 30th International Symposium on Computer Architecture and High Performance Computing

Exploring Self-Adaptivity towards Performance and Energy for Time-stepping Methods

Natalia Kalinnik[1], Robert Kiesel[2], Thomas Rauber[1], Marcel Richter[2] and Gudula Rünger[2]

[1] University Bayreuth
Email: {natalia.kalinnik,rauber}@uni-bayreuth.de

[2] Chemnitz University of Technology
Email: {robert.kiesel,m.richter,ruenger}@cs.tu-chemnitz.de

***Abstract*—Time-stepping simulation methods offer potential for self-adaptivity, since the first time steps of the simulation can be used to explore the hardware characteristics and measure which of several available implementation variants leads to a good performance and energy consumption on the given hardware platform. The version with the best performance or the smallest energy consumption can then be used for the remaining time steps. However, the number of variants to test may be quite large and different simulation methods may require different approaches for self-adaptivity. In this article, we explore the potential for self-adaptivity of several methods from scientific computing. In particular, we consider particle simulation methods, solution methods for differential equations, as well as sparse matrix computations and explore the potential for self-adaptivity of these methods, considering both performance and energy consumption as target function.**

***Index Terms*—autotuning, performance and energy analysis, time-stepping methods.**

I. Introduction

Performance and energy consumption are important aspects for executing parallel application programs on HPC platforms. Porting an existing application program to a new hardware platform may be challenging, especially if the given performance characteristics should be retained or even improved and if the new hardware platform is heterogeneous. Many tuning steps and implementation adaptations may be necessary to meet the desired requirements for performance and energy consumption. The tuning process may be time-consuming, especially if the application is a complex software system that has been developed over many years. In this context, it would be desirable that the runtime behavior of the application program could adapt automatically to the given hardware platform without the need for the application programmer to perform large modifications of the implementation. The ability for adaptation to new systems is often called self-adaptivity or autotuning.

There are several approaches for self-adaptivity for specific applications [1], [2], [3]. There also exist several general autotuning frameworks that are independent from a specific application and aim at a performance improvement for parallel applications from many application areas by an autotuning process [4], [5], [6]. In this article, we propose a tuning methodology which lies between application-specific and fully general tuning. More precisely, self-adaptivity for a specific class of applications with similar computational behavior is considered, which is more general than considering a single application, but less general than considering arbitrary applications.

In particular, the potential for self-adaptivity for time-stepping simulation methods from scientific computing is investigated. The class of time-stepping simulation methods is a broad class of applications, which includes iterative numerical methods as well as physical and chemical simulations. We address the question, which self-adaptivity techniques can be used to support performance and energy tuning for the development of flexible complex software systems. The simulation methods considered include sparse matrix computations, particle simulation methods, and solution methods for ordinary differential equations. The contribution of the article is the provision of a detailed performance and energy analysis of the different simulation methods as well as the investigation of relevant parameters which have a large influence on the performance and energy consumption. Moreover, we analyze whether the parameters identified are amenable to self-adaptivity and which methodology for self-adaptivity is suitable for which of the time-stepping methods. Offline and online techniques for self-adaptivity are considered and it is that, depending on the simulation method and the parameters identified, different approaches for self-adaptivity are best suited for different application programs.

The rest of the article is structured as follows. Section II gives an overview of offline and online autotuning approaches. Section III considers time-stepping applications and discusses their potential for self-adaptivity. Section IV discusses related work and Section V concludes the article.

II. Overview of approaches for self-adaptivity

Application software that facilitates self-adaptivity is able to alter its behavior by providing mechanisms for self-adjustment or self-modification according to changes of the environmental or structural conditions. The term self-adaptivity has already been introduced and investigated

978-1-5386-7769-8/18/$31.00 ©2018 IEEE
DOI 10.1109/SBAC-PAD.2018.00031

Authorized licensed use limited to: Hochschule für Technik Wirtschaft und Kultur. Downloaded on January 19,2026 at 11:05:01 UTC from IEEE Xplore. Restrictions apply.

by [7]. A key component of self-adaptive software is an evaluation mechanism that observes the behavior of the software such that irregularities and non-optimal performance can be recognized. With this information, the software can adjust its behavior to ensure that it is working as intended and may even improve its performance based on changing to more suitable execution modes. This section provides an overview of offline and online mechanisms for self-adaptivity considering time-stepping methods.

A. Tuning parameters and possibilities

The performance of application software may depend on diverse factors to varying degrees. The influencing factors for runtime performance are already well researched for specific applications. However, the optimization of energy consumption or the decision whether certain runtime improvements, e.g. due to a rising number of threads, are sufficient concerning the increased energy consumption may arise as key considerations for future application software. Therefore, we provide a short overview of relevant factors.

A simulation method from scientific computing is performed based on a given implementation. This implementation can be altered by correctness-preserving program transformations, e.g. loop unrolling or loop tiling, SIMD instructions and the consideration of memory prefetchers. The resulting pool of implementation versions can then be used to select implementations suitable to pursue given optimization goals.

In case of loop transformations, relevant factors are for example the unroll factor and the blocking factor for loop unrolling and loop tiling, respectively. In addition to the choice of such transformation parameter values, the order of applied transformations may also have a significant influence on the achieved performance. Accordingly, performance models such as the ECM model [8] could be used to identify appropriate program transformations.

As hardware-specific parameters should be considered as major factors of performance optimization, many tuning parameters have to be taken into account. Therefore, the number and mapping of threads and processes to actual resources of a HPC system, the adaptation of a given implementation to hardware characteristics with appropriate compilers and dynamic voltage and frequency scaling (DVFS) regarding energy consumption in particular constitute crucial factors for self-adaptivity.

Since input data for simulation methods are often the most influential component, their characteristics determine which algorithms are the most efficient ones and how the data should be organized to ensure an optimal processing. Examples are signal processing [2], sparse linear algorithms with sparse input vectors and matrices [9], and particle simulation methods which depend on the input data due to particle movements during the simulation.

B. Offline and online approaches for self-adaptivity

The focus of our investigation is on self-adaptivity for time-stepping simulation methods from scientific computing and the suitable choice of online and offline approaches, which may have varying significance and impact.

1) Offline Autotuning: Offline approaches are used to determine suitable tuning parameter values and implementation versions before the actual execution of the simulation, i.e. within a preceding offline phase. Therefore, main considerations regard invariant properties such as hardware-specific characteristics, available implementation versions and their performance with different classes of application-specific input data, and the limitation of tuning parameter search spaces. In particular, hardware specifications can provide the number and arrangement of available cores, core frequencies, latencies of memory structures and network connections and the availability of accelerator cards, e.g. GPUs. Hence, different implementation versions can be tested with diverse input scenarios to obtain a small set of suitable implementations covering all relevant input scenarios and the given hardware configuration. Moreover, tuning parameters could be limited by reasonable minimum and maximum values, which can be used in an online phase to find an optimal value more efficiently. This process can be supported by performance models such as ECM or the Roofline model [10].

2) Online Autotuning: An online tuning phase is present during the execution of the simulation software and has three major objectives: refine relevant tuning parameters, select the most efficient implementation version and adapt to input data characteristics that may change over time. A key component is the time interval-based evaluation of the actual performance behavior based on runtime and energy measurements. Accordingly, the software uses self-adaptive mechanisms, e.g. to change the implementation version or parameter values, and, thus, adapts its behavior to pursue the given optimization goals as best as possible. The challenge of online approaches is to minimize the impact on the actual simulation performance while integrating the self-adaptation process.

III. Tuning examples and experimental results

The potential for self-adaptivity is investigated with experimental studies considering three diverse application fields, i.e. particle simulation methods, sparse matrix computations, and solution methods for differential equations. Since the runtime and energy behavior may be influenced by several factors to a varying degree, we provide an analysis of a variety of tuning parameters, including the operational frequency, the degree of parallelism, and application-specific parameters, e.g. the maximum tree depth for particle simulation methods.

A. Particle simulation methods

Particle simulations are widely used in various areas of computational science, such as astrophysics, biology and chemistry. The determination of long-range particle interactions such as Coulomb or gravitational interactions is the major effort of the simulation. The interaction

Authorized licensed use limited to: Hochschule für Technik Wirtschaft und Kultur. Downloaded on January 19,2026 at 11:05:01 UTC from IEEE Xplore. Restrictions apply.

of the particles can be simulated with time steps. New positions and velocities for each particle are calculated in the simulation in each time step. However, a direct calculation of the interactions between N particles requires $O(N^2)$ operations. More efficient methods can reduce this complexity with splitting approaches, e.g. Fourier-based methods and hierarchical methods. These approaches split the particle field in near-field and far-field parts. The pairwise interactions between particles are then accurately computed for the near-field part while the far-field part might be computed approximately.

For using the near-field and far-field interactions, the particle field is often decomposed in a grid structure. The near-field interactions are computed inside a grid cell and the far-field interactions computes the interactions of a particle with the particles outside of its grid cell. The size of these cells can be set by parameters and may influence the accuracy of the results of the particle simulation. The major difference of the particle simulation approaches is the computation of the far-field part. In this article, two different approaches are considered, a Fourier-based approach and a hierarchical tree-based approach based on multipole expansions.

The influences on the energy consumption or runtime of the chosen particle simulation methods are examined for both hardware and software parameters. The examined software parameter influences the split of the near-field and far-field parts. Depending on the approach of the method different parameters are used for this separation. The hardware parameters include the processor type and architecture as well as the number of processes used. The input data, i.e, number of particles and the initial distribution of the particles, may also influence the energy consumption and runtime. In the experiments, we concentrate on the influence of the particle system size, the number of processors used, the separation between the near-field and far-field parts, and the resulting grid sizes. This allows for various optimizations where energy-to-solution and time-to-solution needs different parameter configurations. The experiments are performed on an Intel Haswell system with two Xeon E5-2683 v3 processors, each is equipped with 14 cores and has a L3 cache size of 35,840 KB. The performance and energy consumption experiments are performed with two different particle systems, which use a periodic so called cloud-wall system. The main system contains 300 particles, whereas 200 particles are distributed on a regular grid (wall) and 100 particles are distributed randomly (cloud) around the wall. For larger systems, the main system is repeated in every direction resulting in the two different particle systems: the small particle system has $300 \cdot 8^2$ particles and the large particle system has $300 \cdot 8^5$ particles. The particle codes used are from the ScaFaCoS library which uses MPI for the parallelization. The energy was measured using PAPI 5.5.1 and the RAPL interface to read the appropriate MSR registers. The measurements do not include DRAM energy consumption and are repeated 10 times to obtain the shown average values. Intel Turbo Boost is enabled for the experiments.

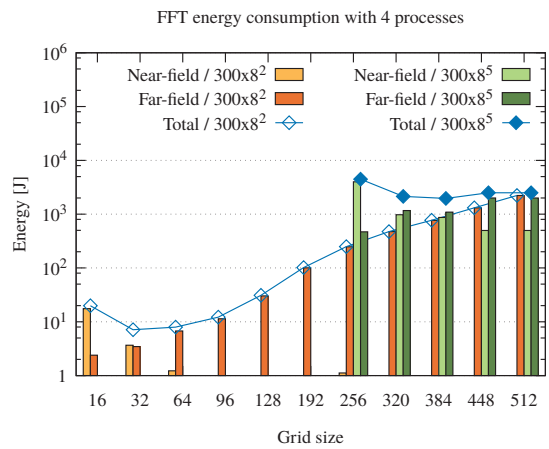

Figure 1. Energy consumption of the FFT-based particle simulation method for different grid sizes for the small and the large particle system for 4 MPI processes.

Fourier-based methods

To compute the far-field of Fourier-based methods, often fast Fourier transforms (FFT) can be used. This approach profits from sufficiently uniformly distributed particles [11]. The separation of the near-field and far-field parts is set by using a specific grid size.

Fig. 1 depicts the resulting energy consumption for both the small and the large particle system for different grid sizes and 4 MPI processes. The grid sizes have a significant influence on the energy consumption. While the energy consumption of the near-field decreases with an increasing grid size, the far field shows an increase. The optimum grid size is achieved when the energy consumption is nearly the same for the near-field and far-field computation. The optimum grid size is 32 for the small particle system for 4 MPI processes. For the large system the optimum grid size is 384 for 4 MPI processes. The development of the execution times are similar to the energy consumption. The optimum grid size for the lowest execution time is 32 for the small particle system for 4 MPI processes but 384 for the large particle system (not shown in a diagram).

Fig. 2 shows the runtime and energy consumption for various numbers of MPI processes for the small particle system. The number of MPI processes has a higher impact on the runtime than on the energy consumption. While the runtime decreases with Hyperthreading, i.e. using 56 MPI processes on the system with 28 cores, the energy consumption increases except for a grid size of 16.

Hierarchical multipole method

Another approach is the fast multipole method (FMM) based on multipole expansions [12]. Unlike the Fourier-based methods, the particles do not have to be uniformly

Authorized licensed use limited to: Hochschule für Technik Wirtschaft und Kultur. Downloaded on January 19,2026 at 11:05:01 UTC from IEEE Xplore. Restrictions apply.

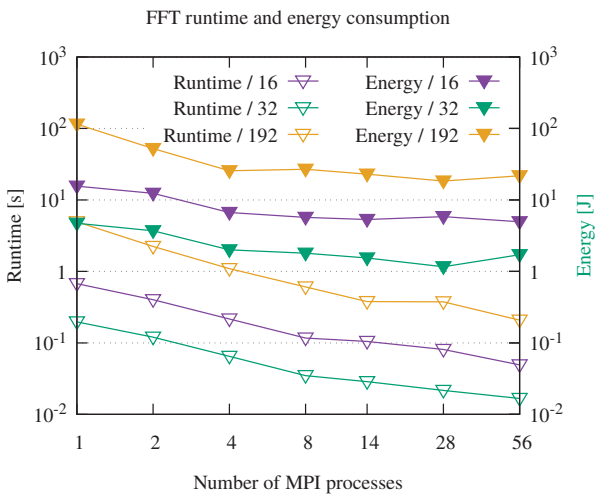

Figure 2. Runtime and energy consumption of the FFT-based particle simulation method for different numbers of MPI processes and grid sizes 16, 32 and 192 for the small particle system.

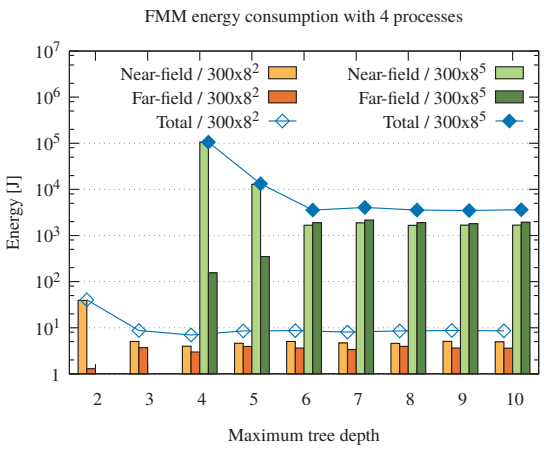

Figure 3. Energy consumption of the FMM particle simulation method for different maximum tree levels for the small and the large particle system using 4 MPI processes.

distributed. The particles are sorted in boxes of a hierarchical octree structure, depending on their position in the particle system. This octree can be influenced by setting a maximum tree depth.

Fig. 3 shows the resulting energy consumption for the small and the large particle system for different maximum tree depths and 4 MPI processes. The maximum tree depth has a significant influence on the energy consumption. The optimum maximum tree depth is 4 for the small particle system and 6 for the large particle system. The runtime has the same behaviour as the energy consumption.

Potential for self-adaptivity

To get the minimal energy consumption or the best performance, mostly the same optimization steps can be chosen for particle simulation methods. There can be small differences for the optimum parameters to get the best energy consumption or performance, e.g. the number of MPI processes. Particle simulation methods require online tuning as the particle positions may change over time, which can lead to a different distribution and, thus, there may be new optimal settings to achieve the lowest energy consumption and the best performance. Nevertheless, since some estimates can be done before the first time step starts, e.g. high MPI process number for the best runtime, offline tuning can help to set acceptable start parameters. Hence, the parameters are initially determined in an offline phase, but must be adjusted over time in an online phase to obtain optimized settings and respond to irregularities.

B. Sparse matrix methods

Sparse matrix operations, e.g. Cholesky factorization, sparse matrix-vector product (SpMV) or sparse general matrix-matrix multiplication (SpGEMM), are key components of many application areas and, thus, require to be performed efficiently with regard to different performance goals. The primary optimization approaches inquired still focus the runtime performance of specific sparse matrix operations. Although runtime performance may prevail as the most significant one, the scope of recent research got more diverse in the past and there are many scientific contributions considering energy consumption as an equally important optimization goal [13].

Hence, sparse matrix operations should be adapted to meet this new optimization challenge and investigate known methods to ensure both optimized runtime and energy consumption. In the case of sparse matrix methods, the major effort should be applied to store sparse matrices efficiently. Therefore, the exploitation of matrix properties, e.g. symmetry or specific patterns of non-zero matrix elements, are used to ensure non-zero matrix elements are stored consecutively and the storage of redundant information, i.e. zero matrix elements, is minimized. The resulting memory layouts for sparse matrices are called sparse matrix formats (SMF). In recent research there can be found numerous SMF [14], each exploiting specific but different matrix properties to ensure efficient processing of matrices of a particular application area.

Further considerations include hardware and software parameters which can be modified and are used to provide an insight of how different SMF perform with regard to a SpGEMM operation on sparse input matrices. The SMFs considered for the comparison are Compressed Sparse Row (CRS), Block Sparse Row (BSR) and Ellpack-Itpack (EIP). The investigated hardware parameters are the number of threads and the core frequency. As a software parameter the block size of the BSR format is chosen and evaluated using runtime and energy measurements. The test matrix is thermal1, which has 574,458 non-zero matrix elements, is symmetric positive definite and based on a thermal problem. This matrix was taken from the SuiteSparse Matrix Collection [15]. The HPC systems used

Authorized licensed use limited to: Hochschule für Technik Wirtschaft und Kultur. Downloaded on January 19,2026 at 11:05:01 UTC from IEEE Xplore. Restrictions apply.

are a Skylake system with two Xeon Gold 6130 processor each with 16 physical cores at 2.1 GHz and a Knights Landing (KL) system with a Xeon Phi 7250 processor with 68 physical cores at 1.4 GHz using the all-to-all mode. Regarding the Skylake system, each physical cores has an own L1 and L2 cache and its own tile. Thus the Skylake system is no MIC architecture. However, the Knights Landing has 34 tiles comprising two physical cores and a shared 1MB L2 cache. In this work MIC was not exploited explicitly but the algorithms were measured up to the maximum number of threads, i.e. 272 threads for the KL system due to the 4 threads per core support. Since the adaptation to MIC processors is quite specific, it may not be a primary optimization goal for an autotuning approach aiming to address many hardware architectures. Thus, it is not addressed in this work but will be taken into account in future work for fine-grained tuning. The parallel sparse matrix methods are realized with OpenMP and C++ and compiled with GCC 4.9.2 with -O2. The energy measurements are done using PAPI 5.5.1 and the RAPL interface to read the appropriate MSR registers. DRAM energy consumption was not considered. The measurements are repeated 12 times and the average values are shown. Intel Turbo Boost is enabled for all experiments.

As shown in Fig. 4 the lowest energy consumption could be achieved with 64 (CRS), 48 (BSR) and 16 Threads (EIP) for the Skylake system. Considering the numbers of threads for the best runtime, the number of threads used increases for BSR to 64 and for EIP to 32 (not shown in the figure). This indicates that different optimization goals, i.e. runtime and energy consumption, require different tuning mechanisms to achieve the performance optimum. The usage of all threads of the Skylake system leads to a significant increase in energy consumption, i.e. 39.5% for CRS and 121.5% for BSR, in comparison to the optimal number of threads stated above. Regarding the KL system, the most energy efficient numbers of threads are 136 (CRS), 272 (BSR) and 32 (EIP), which are also used to achieve the optimal runtime. The increase in energy consumption for a non-optimal number of threads is even more severe compared to the Skylake system, i.e. the EIP format with 272 threads shows an increase in energy of 672% as depicted in the bottom diagram. Although the KL system is 33% to 37% slower than the Skylake system with regard to the runtime measured, the energy consumption actually differs from -13% to 6%. Accordingly, the KL system can achieve the lowest energy consumption for the CRS and BSR format.

Different block sizes for BSR and a varying core frequency is investigated in Fig. 5. Despite the already discussed runtime difference between the two systems, the BSR format performs differently for the block sizes 2 and 16 as shown in the left diagrams. The optimal core frequency is 2.1 GHz for the Skylake system and 1.4 GHz for the KL system. Exceptions are the BSR-2 and the BSR-4 variant, which have an optimal frequency of 2.0 GHz and

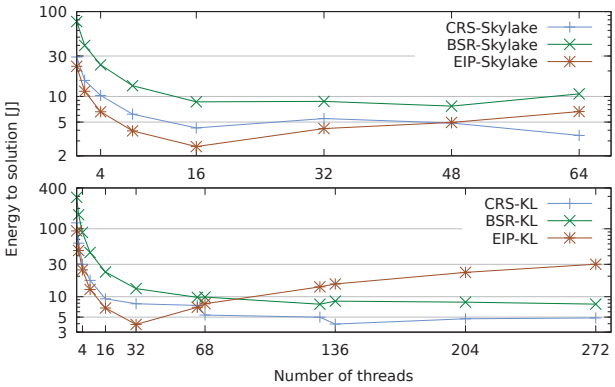

Figure 4. Energy consumption of SpGEMM with three different SMFs and different numbers of threads on a Skylake system (top) and a Knights Landing (KL) system (bottom).

1.3 GHz on the Skylake and KL system, respectively. For the BSR-2 variant this is related to the large number of small blocks, which causes additional overhead regarding the access to the block values, the column and the row information, and, thus, exceed the capabilities of the cache load bandwidth. Therefore, the higher core frequency can not be supported and may also cause additional thread context switches. Regarding the variable core frequency, the expected decrease in runtime is achieved except for a block size of 2 on the KL system. As indicated by the right diagrams of Fig. 5, the block size 2 variant has an increased energy consumption of up to 10% for a non-optimal core frequency regarding the KL system. The optimal core frequency is 1.3 GHz on the KL system and 2.0 GHz for the Skylake system. On the Skylake system, almost all block sizes considered suffer an increase in energy consumption of at least 3% and up to 12% while using the maximum core frequency, i.e. there is no actual benefit for a block size of 8 and a core frequency of 2.1 GHz compared to the minimal frequency of 1.0 GHz in terms of energy consumption.

Potential for self-adaptivity

Regarding sparse matrix methods, the significant parameters are based on properties of the input matrices or hardware properties as shown for the number of threads and the core frequency in the previous section. Since these parameters are invariant for most sparse matrix operations, an offline tuning step is the most promising approach to apply optimizations.

Accordingly, performance models, which rely solely on hardware configurations or static algorithmic properties, known at compile time, can be considered to identify both reasonable values for hardware and software parameters and suitable implementation variants. Examples of beneficial performance models are the Roofline and ECM model. These two models evaluate the significance of specific hardware limitation, e.g. for SIMD processing

Authorized licensed use limited to: Hochschule für Technik Wirtschaft und Kultur. Downloaded on January 19,2026 at 11:05:01 UTC from IEEE Xplore. Restrictions apply.

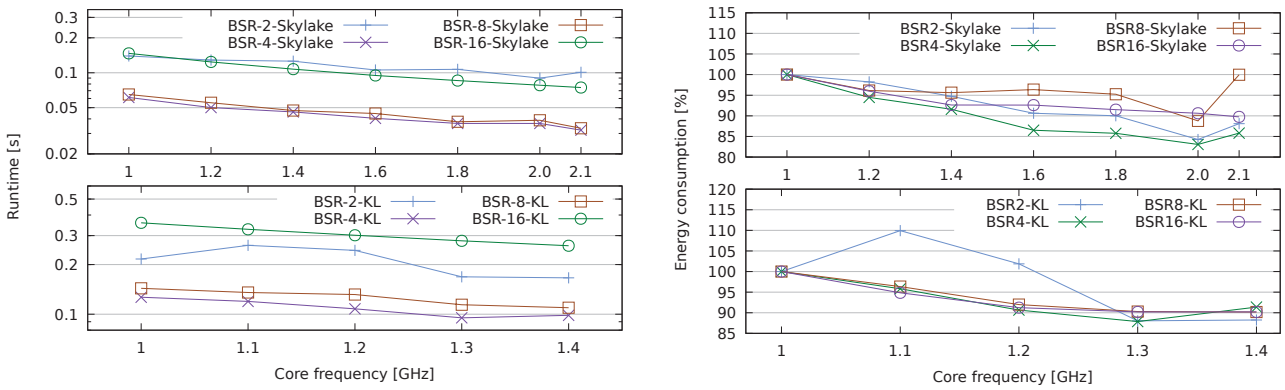

Figure 5. Performance diagrams for SpGEMM with the BSR format considering different block sizes and a variable core frequency using 64 threads. Runtime (left diagrams with logarithmic y-axis) and percental energy consumption (right diagrams), i.e. using the energy measured for the minimum frequency as reference value, are provided for a Skylake system (top) and a Knights Landing (KL) system (bottom).

or memory bandwidth, and, thus, support a selection of suitable implementation variants. Moreover, memory bound algorithms can be analyzed based on invariant cache latencies to predict the optimal number of threads. Hence, performance models can contribute to a great extent to pursue different optimization goals, e.g. runtime or energy consumption.

The potential of online tuning greatly depends on the sparse matrix method performed and the size of the input matrices. Since large matrices may be processed on distributed HPC systems, workload balance and adaptive data distribution schemes can have major impact on the achieved performance [16].

C. Solving differential equations

Numerical solution methods for ordinary differential equations (ODEs) compute an approximate solution for a given ordinary differential equation of the form

$$\mathbf{y}'(x) = \mathbf{f}(x, \mathbf{y}(x)) \text{ with } \mathbf{y}(x_0) = \mathbf{y}_0. \tag{1}$$

by performing a series of time steps one after another until the end of the predefined integration interval is reached [17]. As example method, we consider iterated Runge-Kutta (IRK) methods which perform a fixed number m of computation steps in each time step using the approximation $\mathbf{y}_\kappa$ of the preceding time step. In each computation step, a fixed number s of stage vectors is computed using the stage vectors from the preceding time step and evaluating the function $\mathbf{f}$ defined by the ODE to be solved. After the last computation step, the stage vectors are combined and an approximation $\mathbf{y}_{\kappa+1}$ for the next point in time is computed. An additional approximation of lower order can be computed additionally for error control and for the selection of the step size for the next time step. Overall, a four-dimensional loop structure results within each time step and many typical loop transformations such as loop interchange, loop unrolling, or loop tiling can be applied. Moreover, for many sparse ODE systems, an ordering of the components can be chosen such that the components of the argument vector $\mathbf{y}$ accessed by the function evaluation are located within a limited index range near the component index evaluated. This property of the function $\mathbf{f}$, called limited access distance [18], allows the implementation of special variants by overlapping of vectors and using pipeline-like computational structure of the computation steps m [18].

Taking the parameters of the transformations such as tile sizes and unrolling factors into account, a large number of code variants can be generated and it is not a priori clear, which of these variants will lead to the best performance or to the minimum energy consumption on a given HPC system. This is an ideal situation for autotuning.

Fig. 6 demonstrates that the input data, such as system size and the number of threads, has a large influence on the resulting performance of the multi-threaded implementation variants of the IRK method. In the figure the time per step is plotted against the increasing number of threads. The results are shown for the BRUSS2D test problem and for different system sizes N. The test system BRUSS2D is derived from a reaction diffusion PDE by a spatial discretization on an $N \times N$ grid using the method of lines. The resulting ODE system has dimension $n = 2N^2$. The function $\mathbf{f}$ of BRUSS2D has a limited access distance $2N$, i.e. it accesses only $2N$ components of $\mathbf{y}$ to compute one component of the function result. The experiments have been done on a system with two Intel Xeon E5-2697 v3 processors, each equipped with 14 cores, 35 MB shared cache and running with 2.6 GHz. In the experiments Intel Turbo Boost is enabled. As RK method we use the LobattoIIIC (8) [17] method with $s = 5$ stages and $m = 7$ computation steps. Parallel variants of the ODE methods are developed using pthreads and C, compiled with GCC 4.8.5 and -O3 flag. The code variants [18], [19] exploit parallelism across the system, the n equations of the ODE system are distributed among the available number of threads p. The variants differ in the loop and the

Authorized licensed use limited to: Hochschule für Technik Wirtschaft und Kultur. Downloaded on January 19,2026 at 11:05:01 UTC from IEEE Xplore. Restrictions apply.

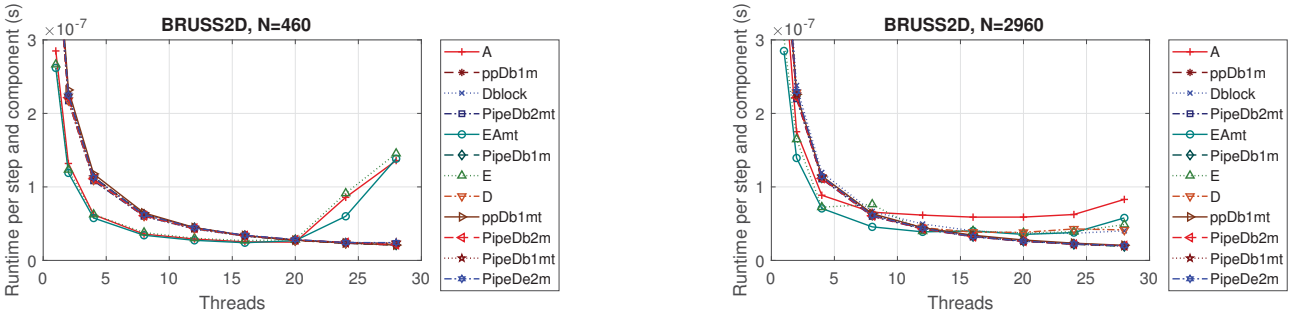

Figure 6. Execution time per step and component of IRK method for $N = 460$ (left) and $N = 2960$ (right) for different numbers of threads.

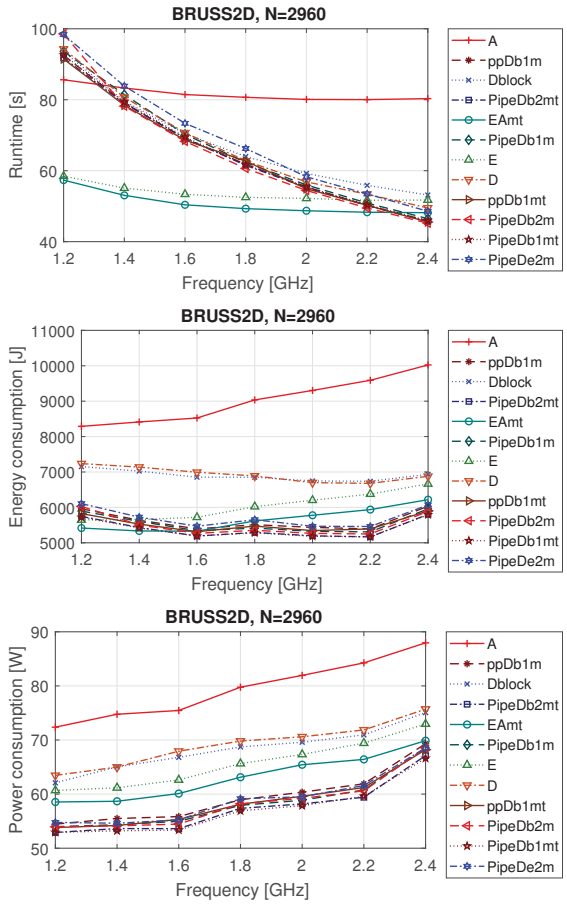

Figure 7. Runtime (top), energy (middle) and power consumption (bottom) of the different variants of the IRK method executed with 16 threads for varying clock frequencies. Test system BRUSS2D with $N = 2960$.

data structures used. This results in different utilization of the cache and the memory hierarchy and consequently leads to different locality behaviour of the variants. The variants with suffix 'mt' and the variant Dblock use loop tiling. The variants A and E use the same data structures, but differ in the order of the loops. In A the innermost loops iterate over the system dimension, all components are accessed with increasing index providing a high spatial locality. E is derived from A by the loop interchange of the two innermost loops: the system dimension and the stage vector loop. This enables temporal reuse of the argument vector components in write operations. The variant Eamt is similar to E, but uses loop tiling. All other variants are derived from the variant D, which results from several loop interchanges and fusions. Further, PipeDb1m, PipeDb1mt, ppDb1m and ppDb1mt variants exploit a limited access distance of the function $\mathbf{f}$ of BRUSS2D. These variants require only scalable neighbor-to-neighbor communication and lock-based local synchronization, whereas all other variants are suitable for ODE problems with function $\mathbf{f}$ with arbitrary access distance, and thus need global communication and barrier operations.

Considering the diagrams in Fig. 6 we notice that the fastest implementation variant depends on the system size and the number of threads executing the program. For the system size $N = 460$ and $p \leq 20$, the variant EAmt delivers the best performance, whereas for $N = 2960$ and thread numbers up to 12, the variant EAmt is the fastest variant. For $N = 460$ and $p > 20$, the variants A, E, EAmt are the slowest implementations, while all other variants obtain similar performance. For $N = 2960$ and $p > 12$, the variants PipeDb1mt and PipeDb1m exhibit the best performance, closely followed by the variant PipeDb2mt. The main reason for the lower performance of the variants A, E and EAmt for the increasing number of threads are high synchronization costs. These variants require two barrier operations per stage in each computation step, whereas all other variants need only one barrier per computation step or use locks for the synchronization of the threads.

Fig. 7 shows the performance, the energy and the power consumption with respect to the clock frequency for the different variants of the IRK method executed with 16 threads. All implementations perform 60 time steps. The energy and the power consumption is measured using LIKWID [20]. The energy measurements do not include DRAM consumption and Turbo Boost is disabled. The test system is Intel Xeon E5-2630 v3, equipped with 16 cores and 20 MB shared L3 cache. The test system is BRUSS2D with $N = 2960$. The results indicate that the best frequency depends on the optimization goal. The

Authorized licensed use limited to: Hochschule für Technik Wirtschaft und Kultur. Downloaded on January 19,2026 at 11:05:01 UTC from IEEE Xplore. Restrictions apply.

variant yielding the best performance for a given frequency is not necessarily the one with minimum energy consumption. When tuning towards the minimum execution time, the best time is achieved at a frequency of 2.4 GHz with the variant PipeDb1mt. While the smallest amount of energy is consumed by one of the variants PipeDb1mt or PipeDb2mt at a frequency of 2.2 GHz. Even though the variants PipeDb1mt and PipeDb2mt are not the fastest variants at this frequency, they show the lowest power usage. The variant leading to the best performance at 2.2 GHz is EAmt, but compared to the PipeDb1mt variant it consumes 15% more power and is only 3% faster.

Potential for self-adaptivity

For ODE methods a pure offline autotuning cannot be applied, since their performance and energy behavior strongly depends on the input data, which is only available at runtime. The access pattern of the function $\mathbf{f}$ of the ODE system to be solved may have a large influence on the resulting performance and the energy consumption of the variants as well as the system size and for parallel implementations the number of threads or processes executing the program. Moreover, the characteristics of the target architecture, such as multi-core processor design, cache architecture, memory latency, network topology and bandwidth have to be taken into account. The choice of the compiler may also affect the runtime and the energy consumption of the variants as well as the alignment of data structures in main memory. Due to the aforementioned factors, ODE methods require online autotuning. On the other hand, the search space of candidate implementations and parameters, e. g. tile sizes for loop tiling and factors for loop unrolling, can be quite large making the application of a pure online autotuning not feasible, since too many code variants would need to be evaluated. Thus, a combination of offline and online autotuning is most promising. Performance models, like ECM, enhanced with offline measurements for the estimation of the synchronization costs can be utilized to predict the execution time of variants or at least to forecast their performance ranking [21]. This information can be used to limit the number of variants evaluated in the online phase. For the optimization towards the energy consumption the analytic energy model presented in [22] can be used in the offline phase to preselect for each implementation variant a set of clock frequencies for which the energy consumption is close to the minimum.

IV. Related work

First basic notions of self-adaptivity were introduced with offline approaches relying on invariant hardware properties of HPC systems and focused execution time. For numerical methods for dense linear algebra problems, well known representatives are ATLAS [1] and PHiPAC [23]. Further investigations of potential techniques to optimize performance and energy consumption of linear algebra algorithms were made by [13]. However, characteristics of input data may imply considerable influence on the execution time and, thus, indicate that offline approaches are not sufficient. Hence, online approaches should be applied to facilitate self-adaptation based on properties of input data. Typical examples of input data dependent problems are signal processing and sparse linear algebra. Accordingly, input data may decide which algorithm is the most efficient one and how data is stored, distributed and processed efficiently. Examples for such approaches are FFTW [2], SPIRAL [3] and OSKI [9]. An approach of autotuning for SpMV and sparse matrix multiplication is presented in [24]. Other self-adaptivity approaches are based on domain-specific languages and utilize compiler generated code versions, which can be evaluated in an online phase [25], [26].

Application independent frameworks that facilitate self-adaptivity such as Active Harmony [4], Parcae [5] and Periscope [6] are often based on compiler approaches. An overview of compiler-based frameworks considering machine learning approaches is presented in [27].

An online autotuning approach to optimize the performance of parallel solution methods of ODEs is proposed in [19]. In this approach the autotuning towards the energy efficiency has not been considered and the number of threads has not been optimized. An autotuning approach for the FMM to optimize its parameters was shown in [28]. Their approach was based on experiments, but did not consider the energy consumption.

Autotuning approaches for minimizing the energy consumption have been investigated in [29], [30]. By means of multi-objective autotuning an application can be optimized for more than one optimization goal simultaneously. Insieme [31] is a compiler framework for multi-objective tuning of parallel programs, it is used in [32] to autotune applications with multiple regions.

V. Conclusions

Self-adaptivity is a property of software to alter its behavior, which requires appropriate autotuning mechanisms. The tuning potential considered in this article can be separated into offline and online approaches. The results show that particle simulation methods require online autotuning, for sparse matrix computations offline autotuning is mainly suitable and for solution methods for differential equations a mixture of both approaches comprises the most potential.

Invariant hardware parameters such as the number of available cores and their adjustable core frequency show a significant impact on runtime and energy consumption for all investigated methods. However, application-specific parameters, input data and diverse implementation versions comprise additional tuning potential which can have a greater influence than the investigated hardware properties. This was shown for varying grid sizes and maximum tree depths for particle simulations, diverse sparse matrix

Authorized licensed use limited to: Hochschule für Technik Wirtschaft und Kultur. Downloaded on January 19,2026 at 11:05:01 UTC from IEEE Xplore. Restrictions apply.

formats and block sizes for sparse matrix multiplication and for ODE methods for varying system sizes. For ODE methods also the order of computations and memory accesses within time steps as well as the access pattern of the problem function **f** have a large influence on the resulting performance and the energy consumption.

Since the significance of tuning approaches, which provide the most benefit, greatly varies between the application methods analyzed, autotuning approaches which support solely either offline or online autotuning may not be sufficient to fully utilize given resources and, thus, miss great potential to further optimize runtime and energy consumption. Hence, tuning mechanisms that may profit from interaction between an offline and online phase can not be utilized to full extent or not at all.

Acknowledgement

This work has been supported by the German Ministry of Science and Education (BMBF), project title SeASiTe (Self-Adaptation of Time-step-based Simulation Techniques on Heterogeneous HPC Systems) with project number 01IH16012A/B.

References

[1] R. Whaley, A. Petitet, and J. Dongarra, "Automated empirical optimizations of software and the ATLAS project," *Parallel Computing*, vol. 27, no. 1-2, pp. 3–35, 2001.

[2] M. Frigo and S. Johnson, "The design and implementation of FFTW3," *Proc. of the IEEE*, vol. 93, no. 2, pp. 216–231, 2005.

[3] M. Püschel, J. Moura, J. Johnson, D. Padua, M. Veloso, B. Singer, J. Xiong, F. Franchetti, A. Gacic, Y. Voronenko, K. Chen, R. Johnson, and N. Rizzolo, "SPIRAL: Code generation for DSP transforms," *Proc. of the IEEE*, vol. 93, no. 2, pp. 232– 275, 2005.

[4] A. Tiwari and J. K. Hollingsworth, "Online adaptive code generation and tuning," in *Proc. of the 2011 IEEE Int. Parallel & Distributed Processing Symp. (IPDPS 2011)*, pp. 879–892, 2011.

[5] A. Raman, A. Zaks, J. Lee, and D. August, "Parcae: A System for Flexible Parallel Execution," in *Proc. of the 33rd ACM SIGPLAN Conf. on Programming Language Design and Implementation*, PLDI '12, pp. 133–144, 2012.

[6] M. Gerndt, E. César, and S. Benkner, eds., *Automatic Tuning of HPC Applications – The Periscope Tuning Framework*. Shaker Verlag, 2015.

[7] J. Aseltine, A. Mancini, and C. Sarture, "A survey of adaptive control systems," *IRE Transactions on Automatic Control*, vol. 6, pp. 102–108, Dec 1958.

[8] J. Hofmann, J. Eitzinger, and D. Fey, "Execution-Cache-Memory Performance Model: Introduction and Validation," *ArXiv e-prints*, Sept. 2015.

[9] R. Vuduc, J. Demmel, and K. Yelick, "OSKI: A library of automatically tuned sparse matrix kernels," in *Institute of Physics Publishing*, 2005.

[10] S. Williams, A. Waterman, and D. Patterson, "Roofline: An insightful visual performance model for multicore architectures," *Commun. ACM*, vol. 52, pp. 65–76, Apr. 2009.

[11] R. W. Hockney and J. W. Eastwood, *Computer Simulation Using Particles*. Bristol, PA, USA: Taylor & Francis, Inc., 1988.

[12] L. Greengard, *The Rapid Evaluation of Potential Fields in Particle Systems*. Boston: MIT Press, 1988.

[13] L. Tan, S. Kothapalli, L. Chen, O. Hussaini, R. Bissiri, and Z. Chen, "A survey of power and energy efficient techniques for high performance numerical linear algebra operations," *Parallel Comput.*, vol. 40, pp. 559–573, Dec. 2014.

[14] D. Langr, I. Simecek, P. Tvrdík, T. Dytrych, and J. P. Draayer, "Adaptive-blocking hierarchical storage format for sparse matrices," *2012 Federated Conference on Computer Science and Information Systems (FedCSIS)*, pp. 545–551, 2012.

[15] T. A. Davis and Y. Hu, "The University of Florida Sparse Matrix Collection," *ACM Trans. Math. Softw.*, vol. 38, no. 1, pp. 1:1–1:25, 2011.

[16] S. Lee and R. Eigenmann, "Adaptive runtime tuning of parallel sparse matrix-vector multiplication on distributed memory systems," in *Proceedings of the 22Nd Annual International Conference on Supercomputing*, ICS '08, pp. 195–204, ACM, 2008.

[17] E. Hairer, S. Nørsett, and G. Wanner, *Solving Ordinary Differential Equations I: Nonstiff Problems*. Springer, 1993.

[18] M. Korch and T. Rauber, "Locality optimized shared-memory implementations of iterated Runge-Kutta methods," in *Euro-Par 2007. Parallel Processing*, vol. 4641 of *Springer LNCS*, pp. 737–747, Springer, 2007.

[19] N. Kalinnik, M. Korch, and T. Rauber, "Online auto-tuning for the time-step-based parallel solution of ODEs on shared-memory systems," *J. of Parallel and Distributed Computing*, vol. 74, no. 8, pp. 2722–2744, 2014.

[20] J. Treibig, G. Hager, and G. Wellein, "LIKWID performance tools," in *Competence in High Performance Computing* (C. Bischof *et al.*, eds.), 2010.

[21] J. Seiferth, C. Alappat, M. Korch, and T. Rauber, "Applicability of the ECM Performance Model to Explicit ODE Methods on Current Multi-core Processors," in *High Performance Computing*, pp. 163–183, Springer, 2018.

[22] T. Rauber, G. Rünger, and M. Stachowski, "Model-based optimization of the energy efficiency of multi-threaded applications," in *Proc. of the 8th Int. Green and Sustainable Comp. Conf., IGSC*, pp. 1–6, 2017.

[23] J. Bilmes, K. Asanovic, C. Chin, and J. Demmel, "Optimizing Matrix Multiply Using PHiPAC: A Portable, High-performance, ANSI C Coding Methodology," in *Proc. of the 11th Int. Conf. on Supercomputing*, ICS '97, pp. 340–347, 1997.

[24] B. C. Lee, R. W. Vuduc, J. W. Demmel, and K. A. Yelick, "Performance models for evaluation and automatic tuning of symmetric sparse matrix-vector multiply," in *Int. Conf. on Parallel Processing*, pp. 169–176 vol.1, Aug 2004.

[25] M. Christen, O. Schenk, and H. Burkhart, "PATUS: A code generation and autotuning framework for parallel iterative stencil computations on modern microarchitectures," in *Proc. of the 25th IEEE Int. Parallel and Distributed Processing Symp.*, 2011.

[26] Y. Tang, R. A. Chowdhury, B. C. Kuszmaul, C.-K. Luk, and C. E. Leiserson, "The Pochoir stencil compiler," in *Proc. of the Twenty-third Annual ACM Symp. on Parallelism in Algorithms and Architectures (SPAA '11)*, pp. 117–128, 2011.

[27] A. H. Ashouri, G. Palermo, J. Cavazos, and C. Silvano, *Automatic Tuning of Compilers Using Machine Learning*. Springer Publishing Company, Incorporated, 1st ed., 2017.

[28] H. Dachsel, M. Hofmann, J. Lang, and G. Rünger, "Automatic tuning of the fast multipole method based on integrated performance prediction," in *Int. Conf. on High Performance Computing and Communication (HPCC'12)*, pp. 617–624, IEEE, 2012.

[29] A. Tiwari, M. A. Laurenzano, L. Carrington, and A. Snavely, "Auto-tuning for energy usage in scientific applications," in *Proc. of the 2011 Int. Conf. on Parallel Processing*, Euro-Par'11, pp. 178–187, Springer, 2012.

[30] A. Panyala, D. Chavarra-Miranda, J. B. Manzano, A. Tumeo, and M. Halappanavar, "Exploring performance and energy tradeoffs for irregular applications," *J. Parallel Distrib. Comput.*, vol. 104, pp. 234–251, June 2017.

[31] P. Gschwandtner, J. J. Durillo, and T. Fahringer, "Multi-objective auto-tuning with insieme: Optimization and trade-off analysis for time, energy and resource usage," in *Euro-Par 2014 Parallel Processing*, pp. 87–98, Springer, 2014.

[32] K. Kofler, J. J. Durillo, P. Gschwandtner, and T. Fahringer, "A region-aware multi-objective auto-tuner for parallel programs," in *46th Int. Conf. on Parallel Processing Workshops (ICPPW)*, pp. 190–199, Aug 2017.

Authorized licensed use limited to: Hochschule für Technik Wirtschaft und Kultur. Downloaded on January 19,2026 at 11:05:01 UTC from IEEE Xplore. Restrictions apply.